



Parere

GRC

Gobernanza, Riesgo y Cumplimiento

DOCUMENTACIÓN DE SOFTWARE - SENA ADSO

MANUAL DE DESPLIEGUE

Guía paso a paso para configurar la infraestructura y publicar el sistema en la nube.

PROGRAMA DE FORMACIÓN
Análisis y Desarrollo de Software (ADSO)
TECNOLOGÍA

AUTORES / DESARROLLADORES
Adriana Barrios
Juan Cadena

TABLA DE CONTENIDO

1.	PROPÓSITO DEL PROYECTO	3
2.	ARQUITECTURA GENERAL	3
3.	INSTRUCCIONES DE DESPLIEGUE EN LA NUBE	4
3.1.	Despliegue de Base de Datos (Clever Cloud)	4
3.1.1	Primer despliegue (clean Install)	4
3.2.	Despliegue de Backend (Render).....	5
3.2.1.	Configuración Dockerfile	6
3.3.	Despliegue de Frontend (Cloudflare Pages)	6
4.	Procedimiento de Verificación Post-Despliegue (<i>Smoke Tests</i>).....	7
4.1.	Test 1 (Conectividad Backend)	7
4.2.	Test 2 (Prueba de CORS e Integración con el Frontend - Consola DevTools).....	7
4.3.	Test 3 (Prueba de Autenticación / POST (Login))	8

1. PROPÓSITO DEL PROYECTO

Parere GRC es una solución tecnológica integral de Gobernanza, Riesgo y Cumplimiento (GRC) diseñada para centralizar, agilizar y asegurar todo el ciclo de vinculación comercial con terceros de la organización.

A través de esta herramienta, los proveedores pueden autogestionar su postulación mediante un flujo guiado que incluye la carga de soportes legales, financieros y tributarios, asistida por el procesamiento automático de documentos, lo que permite minimizar la digitación y los errores humanos. Al mismo tiempo, el equipo interno de compras, auditoría y control de riesgos cuenta con un entorno centralizado para cotejar la información campo por campo, calificar la solvencia y emitir dictámenes de aprobación o rechazo.

Finalmente, la plataforma formaliza la relación comercial mediante mecanismos de firma electrónica y la generación de certificados oficiales, manteniendo una trazabilidad y una auditoría completas de cada cambio o decisión tomada a lo largo del proceso.

2. ARQUITECTURA GENERAL

El proyecto sigue una arquitectura distribuida tradicional cliente-servidor (monolito frontend + monolito backend RESTful API).

- Frontend: Provee todas las vistas, desde el registro inicial de proveedores hasta los paneles de control internos.
 - Tecnología: HTML5, CSS3, Vanilla JavaScript.
 - Despliegue recomendado: GitHub + Cloudflare Pages. Al ser archivos puramente estáticos (`.html`, `.css`, `.js`), Cloudflare Pages se conecta directamente al repositorio de GitHub y despliega la página web de forma gratuita, rápida y con certificado SSL (HTTPS) automático.
- Backend: Contiene los controladores y la lógica para la evaluación de riesgos, el manejo de notificaciones, los usuarios y la autenticación.
 - Tecnología: Java 17 con framework Spring Boot 3
 - Despliegue recomendado: GitHub + Render. Render es una plataforma en la nube (PaaS) que se conecta a GitHub, compila automáticamente el código de Spring Boot usando Maven y expone la API RESTful y cuenta con una capa gratuita.
- Base de Datos
 - Tecnología: MySQL 8 (en la nube, mediante el servicio de Clever Cloud).
 - Modelo: Relacional. Las tablas se autogeneran e inician mediante Spring Data JPA, con Hibernate como motor ORM.

- Conexión: Se realiza mediante JDBC (*Java Database Connectivity*). En Render, se configuran las credenciales como variables de entorno para proteger la seguridad.
- Librerías / Dependencias Externas
 - Lombok: Para reducir el boilerplate en Java.
 - Spring Data JPA & MySQL Connector: Para la orquestación de la DB.
 - Java Mail Sender: Para envío de notificaciones por correo (SMTP).
 - Apache PDFBox: Para la lectura, validación y manipulación de documentos PDF.
 - OpenPDF: Para la generación dinámica y creación de documentos y certificados en formato PDF.
 - Librerías Frontend: Fuentes de Google Fonts y FontAwesome (cargadas vía CDN - Content Delivery Network).

3. INSTRUCCIONES DE DESPLIEGUE EN LA NUBE

Se dan las instrucciones para los despliegues automatizados conectados directamente a GitHub.

3.1. Despliegue de Base de Datos (Clever Cloud)

- Crea una cuenta en un proveedor de bases de datos relacionales, como **Clever Cloud** que es compatible con MySQL.
- Crea un servicio de MySQL 8+.
- El proveedor te entregará unas credenciales: `Host` (URL), `Port` (3306), `Database Name`, `User` y `Password`. Es importante guardar estos datos.

3.1.1 Primer despliegue (clean Install)

La primera vez que el backend se conecte a esta base de datos, Hibernate creará automáticamente todas las tablas en blanco. Posteriormente, se debe conectar a DBeaver y ejecutar el script 01_seed_datos_iniciales.sql. Esto inyectará los catálogos base y un administrador temporal (admin@parere.com) para arrancar la parametrización.

Nota. En caso de que se requiera el archivo 'create_tables.sql', que contiene la estructura de la base de datos.

3.1.2 Actualización

Si se trata de una actualización del entorno de producción que ya contiene datos, SIEMPRE realiza un volcado (Dump) completo de la base de datos, previo a la actualización.

- Abre DBeaver: Conecta la base de datos con las credenciales de Clever Cloud.
- Ejecuta el script (restauración):
 - Haz clic derecho sobre tu base de datos en Clever Cloud
 - Ir a Herramientas > Restaurar Base de Datos (o Ejecutar script SQL)
 - Selecciona el archivo .sql (el volcado)
 - Selecciona iniciar. DBeaver leerá las líneas del archivo SQL y empezará a insertar todos los usuarios, contraseñas, roles y evaluaciones exactamente como estaban en la base de datos anterior.
- Revisa el Spring Boot

3.2. Despliegue de Backend (Render)

- Ingresa a Render.com y vincula tu cuenta de GitHub, donde se encuentra el repositorio con el código.
- Crea un nuevo "Web Service" y selecciona el repositorio del proyecto.
- Configura el entorno de ejecución:
 - Language: Docker
 - Branch: main
 - Region: US West
 - Root directory: Backend/proveedores/proveedores
- En la sección "Environment Variables" en Render se agregan las credenciales de la base de datos y el correo electrónico bajo los siguientes parámetros:
 - Variable: EMAIL_USER | Value: pareregrc@gmail.com
 - Variable: EMAIL_PASSWORD | Value: (Tu contraseña para app Gmail)
 - Variable: DB_URL | Value: jdbc:mysql://bx3fcwhyvrme9gt1y86m-mysql.services.clever-cloud.com:3306/bx3fcwhyvrme9gt1y86m (se debe ajustar a formato JDBC la entrega por Clever)
 - Variable: DB_USER | Value: (Tu usuario de Clever Cloud)
 - Variable: DB_PASSWORD | Value: (Tu contraseña de Clever Cloud)
- Haz clic en **Deploy**. Render compilará el Java y entregará una URL pública para la API (ej. `https://parere-backend.onrender.com`).

3.2.1. Configuración Dockerfile

El backend utiliza una compilación empaquetada con Docker. El repositorio contiene en su raíz el archivo Dockerfile con la siguiente estructura de ejecución:

```
# Etapa de construcción: descarga las dependencias de Maven y genera el
ejecutable .jar omitiendo los test unitarios para agilizar el despliegue.
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
# Compilamos el proyecto omitiendo los tests para mayor velocidad
RUN mvn clean package -DskipTests

# Etapa de ejecución: utiliza una imagen ligera de JRE de Java 17 para reducir
el consumo de memoria y optimizar el tiempo de arranque en el servidor de
Render.
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
# Copiamos el .war o .jar generado desde la etapa de construcción
COPY --from=build /app/target/proveedores-0.0.1-SNAPSHOT.jar app.jar
# Exponemos el puerto estándar
EXPOSE 8080
# Comando para iniciar la app
ENTRYPOINT ["java", "-jar", "app.jar"]
```

3.3. Despliegue de Frontend (Cloudflare Pages)

- Antes de subir al repositorio de GitHub, abre el archivo config.js y en la configuración reemplaza la variable `API_BASE_URL`, por la URL pública que creó Render (ej. `https://parere-backend.onrender.com/``).

```
const CONFIG = {
  API_BASE_URL: 'https://proyectoadso.onrender.com'
};
```
- Ingresa a Cloudflare, en el menú Workers & Pages, selecciona Create Application > Pages.
- Conecta Cloudflare con tu repositorio de GitHub y selecciona el repositorio (ej. `ProyectoAdso``).
- En la configuración "Build settings":

- Framework preset: `None`
- Build command: (Déjalo vacío)
- Build output directory: `/` (o la ruta donde esté el archivo `index.html`).
- Haz clic en Deploy. Cloudflare clonará el repositorio y proporcionará una URL pública y segura con HTTPS para acceder al proyecto. (ej. pareregrc.pages.dev)

4. Procedimiento de Verificación Post-Despliegue (*Smoke Tests*)

4.1. Test 1 (Conectividad Backend)

Esta prueba confirma que el servicio en Render está encendido y que el Backend se conecta exitosamente a MySQL para consultar datos.

1. Abre una nueva pestaña en tu navegador e ingresa a cualquiera de estos enlaces:
<https://proyectoadso.onrender.com/roles>
<https://proyectoadso.onrender.com/tipo-identificacion>
2. Resultado esperado: Debes ver en pantalla un objeto JSON, similar a:

```
{
  "data": [
    { "id": 1, "nombre": "Administrador", "descripcion": "Administrador del sistema", "activo": true },
    ...
  ]
}
```

4.2. Test 2 (Prueba de CORS e Integración con el Frontend - Consola DevTools)

Esta prueba valida que el Frontend pueda consumir la API sin bloqueos de seguridad de CORS.

1. Abre el Frontend en el navegador.
2. Presiona F12 para abrir las **Herramientas de Desarrollador** y ve a la pestaña **Consola** (Console).
3. Desbloquea la consola: escribe **allow pasting** y presiona Enter
4. Pega el siguiente código y presiona Enter:

```
fetch(CONFIG.API_BASE_URL + '/roles')
  .then(response => {
    console.log('Status HTTP:', response.status);
    return response.json();
  })
```

```

.then(data => {
  console.log('✅ Conectividad Exitosa:', data);
})
.catch(error => {
  console.error('❌ Error de Conectividad o CORS:', error);
});

```

5. Resultado esperado: Ver en la consola Status HTTP: 200 y ✅ Conectividad Exitosa: con la lista de roles.

4.3. Test 3 (Prueba de Autenticación / POST (Login))

1. Valida que las peticiones POST procesen los parámetros y verifiquen a los usuarios en la base de datos. En la misma consola del navegador ejecuta:

```

fetch(CONFIG.API_BASE_URL + '/usuarios/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    usuario: 'TU_CORREO_REAL_AQUI',
    contraseña: 'TU_CONTRASENA_REAL_AQUI'
  })
})
.then(res => res.json())
.then(data => console.log('Resultado Login:', data))
.catch(err => console.error('Error:', err));

```

2. Resultado esperado:

```
{data: {...}}
```

```

data: {successful: true, message: 'Login
exitoso', idUsuario: 2, nombreUsuario: 'Adriana
Barrios', correoUsuario: 'adbarrios87@gmail.com', ...}

```

```
[[Prototype]]: Object
```